HƯỚNG DẪN THỰC HÀNH

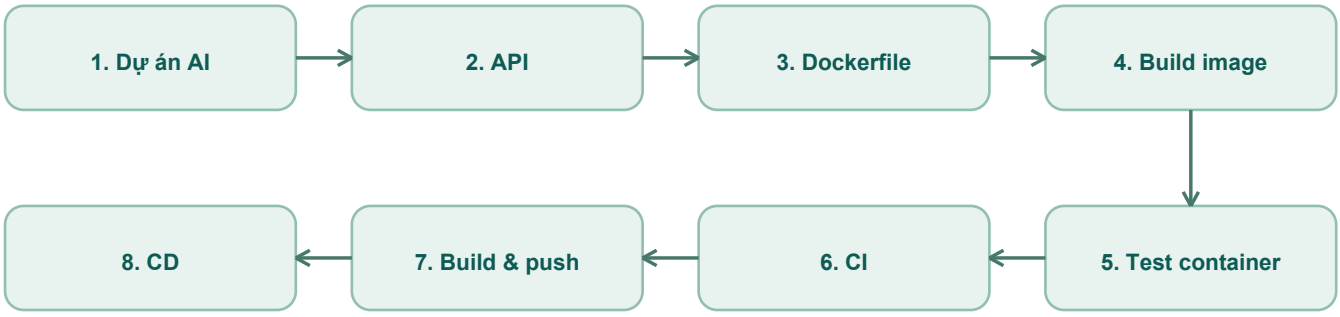
Triển khai VoiceScope AI theo 8 bước

Hướng dẫn thực hành | Đối chiếu kết quả đã kiểm chứng ngày 31/10/2026

Quy trình bạn đề xuất phù hợp: chuẩn bị dự án, tạo API, đóng gói Docker, kiểm thử, thiết lập CI, tự động phát hành image và triển khai lên server. Đây là thứ tự thiết lập lần đầu; sau đó CI tự lặp lại các bước build và kiểm thử mỗi khi mã thay đổi.

Phạm vi: phục vụ inference CPU của VoiceScope bằng model NII AntiDeepfake ONNX Q4 có sẵn, không huấn luyện lại. Mã nguồn: https://github.com/nqming/voicescope. Web đã kiểm chứng: https://voicescope-581u.onrender.com.

Trạng thái: CI/build/push Docker Hub và CD Render đã tự động hóa và được kiểm chứng đầu cuối. Render theo dõi main, chờ CI đạt rồi tự deploy; bản mới đã Live và qua kiểm tra model/API. PDF giữ đúng 8 bước và ghi nhận bằng chứng thực tế.



Thiết lập lần đầu theo 8 bước; CI sẽ tự lặp lại build và kiểm thử.

Mỗi bước có mục đích, file liên quan, thao tác, kết quả mong đợi và bằng chứng đã thực hiện. Lệnh dùng PowerShell; khi không ghi khác, chạy từ thư mục gốc checkout sạch của dự án. Các lệnh hướng dẫn không có nghĩa vừa được chạy lại trong lần biên soạn tài liệu này.

1. Chuẩn bị dự án AI

Mục đích. Tập hợp mã nguồn, weights/checkpoint, dependencies và cấu hình để ứng dụng có thể được tái lập trước khi triển khai.

File liên quan. voicescope/ chứa API và xử lý audio; web/ chứa giao diện; configs/default.json chứa cấu hình. requirements-runtime.txt pin dependencies API; requirements-ci.txt thêm công cụ test. tools/export_antideepfake.py chuẩn bị model; MODEL-NOTICE.md và licenses/ ghi nguồn và giấy phép.

Chuẩn bị môi trường. Cần Git, Python 3.12, Docker chạy Linux containers; Node.js dùng cho test PCM. Máy đã có dự án có thể dùng checkout sạch hiện tại. Nếu cần clone mới, chạy từ thư mục cha bạn chọn và dùng tên thư mục chưa tồn tại:

```
git clone https://github.com/nqming/voicescope.git voicescope-project
Set-Location voicescope-project
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-ci.txt
docker version
```

Thay voicescope-project bằng tên thư mục khác nếu đã có. Sau Set-Location, các bước tiếp theo chạy tại đây. docker version phải hiển thị cả Client và Server; nếu chỉ có Client, khởi động Docker Desktop và chọn Linux engine.

Checkpoint/model. Dự án dùng NII Yamagishi Lab wav2vec-small-anti-deepfake, revision 9a13264b5dcc827a8d5a4f8e01fccefa392f886b. Khi build, exporter tải weights công khai, kiểm SHA256, xuất ONNX và lượng tử hóa Q4 với activation FP32. Thư mục model trong image gồm model_q4.onnx, manifest.json, model card và license. Weights Q4 khoảng 90 MB; không cần GPU/CUDA.

Điều kiện sử dụng. Model có giấy phép CC BY-NC-SA 4.0, phục vụ nghiên cứu/giáo dục phi thương mại và giữ attribution, license, mô tả thay đổi. Không đưa audio riêng, dataset, token hoặc checkpoint không được cấp quyền vào repository hay image.

Kiểm tra hoàn tất. Source và requirements có đủ, môi trường chạy được, nguồn model/revision/quyền phân phối rõ ràng. Bản phát hành đã kiểm dùng weights có sẵn; không có lượt train lại nào trong quy trình này.

2. Xây dựng API

Mục đích. Cung cấp giao diện HTTP để web hoặc chương trình khác gửi audio vào model và nhận kết quả suy luận.

File liên quan. voicescope/api.py khai báo FastAPI; voicescope/server.py khởi động theo HOST/PORT; voicescope/antideepfake_inference.py tải ONNX và suy luận. Giao diện web được phục vụ cùng API, không cần một server frontend riêng.

Luồng xử lý. Audio WAV/FLAC được kiểm giới hạn và giải mã, chuyển mono 16 kHz, chuẩn hóa và chia cửa sổ. Model trả điểm fake; API tổng hợp thành nhãn real/fake và JSON để web hiển thị. Inference không cập nhật weights.

Chạy API từ gốc dự án:

```
.\.venv\Scripts\python.exe -m voicescope.server
```

Mở http://127.0.0.1:8000/docs để xem OpenAPI. GET /health và /api/status báo trạng thái; GET /ready trả 200 khi model sẵn sàng, 503 khi chưa sẵn sàng. POST /inference nhận bytes audio trực tiếp, không nhận multipart form.

Gọi dự đoán khi model đã sẵn sàng:

```
curl.exe --fail-with-body -X POST `
  -H 'Content-Type: audio/wav' `
  --data-binary '@sample.wav' `
  http://127.0.0.1:8000/inference
```

Thay sample.wav bằng WAV bạn được phép xử lý; với FLAC dùng Content-Type: audio/flac. JSON kết quả có label, fake_score, threshold, model và timing_ms.

Kiểm tra hoàn tất. /health trả 200 chưa đủ để kết luận inference hoạt động: cần model_ready=true, /ready 200 và một POST /inference thành công. API native chưa có ONNX/weights có thể trả 503; cách nạp model tự chứa cho người mới là image được build ở bước 3-5.

Lỗi cần nhận biết. 415 thường do Content-Type; 422 do audio không hợp lệ/quá ngắn/im lặng; 429 do rate limit hoặc đang xử lý yêu cầu khác. Dừng server native bằng Ctrl+C trước khi dùng cùng cổng cho container.

3. Tạo Dockerfile

Mục đích. Mô tả môi trường, dependencies, model và lệnh khởi động để build một gói ứng dụng chạy nhất quán trên máy có Docker.

File liên quan. Dockerfile, .dockerignore, requirements-runtime.txt, tools/export_antideepfake.py, MODEL-NOTICE.md và licenses/.

Thiết kế đang dùng. Base Python 3.12 slim được pin digest. Stage runtime cài API và tạo user UID 10001. Stage antideepfake-build dùng PyTorch để xuất model ONNX/Q4. Stage cuối web chỉ giữ runtime ONNX, source, giao diện và model; không chứa PyTorch hoặc checkpoint nguồn FP32 của builder.

Trích phần cấu hình stage cuối trong Dockerfile hiện có:

```
FROM runtime AS web
ARG VCS_REF=local
ENV VOICESCOPE_VERSION=$VCS_REF
USER root
RUN python -m pip install --no-cache-dir 'onnxruntime==1.20.1'
COPY --from=antideepfake-build /opt/export /opt/models/antideepfake
ENV VOICESCOPE_MODEL=antideepfake-int8
USER appuser
```

Đây là trích đoạn để đọc hiểu, không phải toàn bộ Dockerfile để ghi đè. Các stage trước, OCI labels và bản license đầy đủ vẫn nằm trong file thật. CMD kế thừa từ runtime chạy python -m voicescope.server; server bind 0.0.0.0 và đọc PORT.

Tên selector antideepfake-int8 là tên tương thích cấu hình cũ; weights đang phục vụ thực tế là Q4 với activation FP32. Target triển khai được chọn là web.

Build context. .dockerignore dùng default-deny và chỉ cho các file cần thiết. data/, tmp/, artifacts/, .git, .env, audio, notebook và PDF không được gửi vào build context. Weights công khai được tải theo revision cố định trong builder, không lấy từ dataset hay thư mục ghi âm của bạn.

Kiểm tra hoàn tất. Dockerfile có startup rõ ràng, user không root, model được cung cấp có phiên bản/checksum/license; không dùng ARG cho secret. Dockerfile chỉ là công thức build, chưa phải image chạy được. Bước 4 mới tạo image và bước 5 mới xác minh container thực sự hoạt động.

4. Build Docker Image

Mục đích. Thực thi Dockerfile để đóng gói ứng dụng AI thành image gồm môi trường chạy, web/API và model đã chuẩn bị.

File và thư mục. Dùng Dockerfile, .dockerignore và source tại gốc checkout sạch. Docker Desktop phải đang chạy Linux engine; build cần mạng đến các nguồn dependency/model công khai.

```
git status --short
$revision = git rev-parse HEAD
docker build --target web --build-arg VCS_REF=$revision `
  --tag voicescope:local --progress plain .
docker image inspect voicescope:local `
  --format '{{.Config.User}} {{.Size}}'
```

git status --short nên không có thay đổi source chưa commit. Nếu build source chưa commit, dùng VCS_REF=workspace thay vì gán SHA của code cũ. voicescope:local là tag cục bộ, có thể đổi tên; dấu chấm cuối lệnh chỉ build context là thư mục hiện tại.

Điều gì xảy ra? Docker lấy base image, cài dependencies, tải weights đã pin và kiểm checksum, xuất ONNX Q4 rồi tạo stage web. Những lớp không đổi có thể dùng cache. Artifact builder không tự đi vào stage cuối nếu không được COPY.

Kết quả mong đợi. Lệnh build kết thúc thành công và docker image inspect tìm thấy image. Image chưa tự khởi động server; phải docker run ở bước 5.

Bằng chứng đã kiểm. Bản public v0.2.0 là linux/amd64, khoảng 795 MiB; model Q4 khoảng 86 MiB. Một image đã build local và image do CI build là hai artifact riêng nếu khác metadata hay môi trường. Digest registry ở bước 7 xác định đúng artifact được publish.

Giới hạn tái lập. Base và dependencies chính đã pin, model nguồn có checksum. Builder chưa có lockfile kèm hash cho toàn bộ dependency bắc cầu, nên không cam kết rebuild khác thời điểm giống từng byte. Muốn chạy đúng gói đã kiểm, dùng digest đã publish.

Lỗi đã xử lý. Build đầu từng lỗi HTTP 403 khi tải văn bản license; đã lưu license gốc trong licenses/ và COPY vào image. Nếu Docker API không kết nối, kiểm Engine thay vì bỏ qua build.

5. Chạy thử Container

Mục đích. Xác minh image thực sự khởi động, tải model và chạy inference. Image là gói đóng sẵn; container là phiên chạy của gói đó.

File liên quan. tools/container_smoke.py kiểm readiness và suy luận; tools/audit_image.py kiểm payload/model; voicescope/smoke.py kiểm web/API qua HTTP. Chạy từ gốc dự án sau bước 4, đã dừng server native dùng cổng 8000.

```
docker run -d --name voicescope-test `
  --read-only --tmpfs /tmp:size=64m,mode=1777 `
  --cap-drop ALL --security-opt no-new-privileges `
  -p 127.0.0.1:8000:8000 voicescope:local
python tools/container_smoke.py
python tools/audit_image.py --image voicescope:local
```

Các lệnh python kiểm tra dùng Python của môi trường đã chuẩn bị; nếu chưa kích hoạt .venv, thay python bằng .\.venv\Scripts\python.exe. Nếu cổng/name đã dùng, chọn cổng/name khác và truyền --url tương ứng cho smoke.

Cần quan sát. /ready 200, model_ready=true, trang chủ và tài nguyên JS truy cập được; /inference trả score hữu hạn trong [0,1] và nhãn hợp lệ. Audit yêu cầu UID khác 0, không có torch/audio/secret/cache Python trong payload /app và weights đúng checksum manifest.

Smoke mặc định sinh WAV sin 2 giây trong RAM để kiểm đường thực thi. Câu TTS tổng hợp dài 5,368 giây cũng đã được kiểm: trả fake, fake_score 0,99997365. Cả hai không phải bằng chứng accuracy tổng quát và không thay thế đánh giá held-out tiếng Việt.

Kết quả đã thực hiện. Container local và container pull từ Docker Hub đều khởi động, chạy model và qua audit. Bản ghi microphone riêng không được dùng để kiểm trên server public.

Dọn đúng container thử sau khi xong:

```
docker stop voicescope-test
docker rm voicescope-test
```

Lệnh trên không xóa image hay dữ liệu của bạn. Xem docker logs voicescope-test trước khi dọn nếu có lỗi startup. Không coi /health 200 là đủ khi /ready còn 503.

6. Thiết lập CI

Mục đích. Tự kiểm mã nguồn và khả năng build/chạy mỗi khi có thay đổi, phát hiện lỗi trước khi phát hành.

File liên quan. .github/workflows/ci-docker.yml, requirements-ci.txt và tests/. Workflow có job tests và job image chỉ chạy sau khi tests đạt.

Nội dung kiểm. Job tests cài Python 3.12 và Node.js, chạy pytest và test PCM. Job image build target web trên linux/amd64, chạy hardened container, gọi inference và audit payload. Các báo cáo JSON được lưu thành artifact container-evidence.

Có thể chạy các kiểm tra tương ứng tại gốc dự án:

```
.\.venv\Scripts\python.exe -m pytest -q
node tests/test_pcm_capture.cjs
```

Kích hoạt. Tạo PR thay đổi mã nguồn hoặc push main/release/**; version tag v* cũng kích hoạt. PR chỉ sửa docs/README/trình render PDF được bỏ qua theo paths-ignore. Vào GitHub > Actions > CI and Docker Hub để đọc từng job/step. Không cần tự build trên máy cá nhân rồi upload log để giả làm CI.

PR chạy kiểm tra/build, không nhận secrets publish. Login/push chỉ được xét sau kiểm thử, với event push trên các nhánh/tag được tin cậy. Workflow dùng quyền contents: read và pin các Action chính theo commit SHA.

Kết quả đã kiểm. Môi trường Windows: 28 passed, 1 skipped; Linux CI kiểm chứng tự động: 26 passed, 2 skipped; PCM test pass. Các test ML tùy chọn khác nhau theo môi trường. Đây là kiểm chức năng, không phải accuracy của detector.

Lần phát hành v0.2.0 đã qua CI: https://github.com/nqming/voicescope/actions/runs/41185814386. Lượt kiểm chứng auto-CD sau bản sửa phiên bản: https://github.com/nqming/voicescope/actions/runs/41189828565. Nếu job thất bại, đọc step đầu tiên lỗi và sửa nguyên nhân trước khi phát hành. CI đầu từng thiếu file test PCM trên nhánh deploy; đã đồng bộ file thay vì bỏ qua test.

Điểm cần hiểu. CI ở bước này tự lặp lại build và chạy thử của bước 4-5. Thứ tự tám bước là quy trình thiết lập ban đầu, không phải mỗi lần sửa code bạn phải tự làm lại mọi bước bằng tay.

7. Tự động build và push Image

Mục đích. Đưa đúng image đã qua CI lên Container Registry để máy khác hoặc server có thể pull và chạy mà không tự cài từng dependency.

File/cấu hình. Workflow ci-docker.yml và tools/publish_image.py. Trong GitHub > Settings > Secrets and variables > Actions, đã có variable DOCKERHUB_USERNAME=qminh19 và secret DOCKERHUB_TOKEN. Token được lưu mã hóa; không đặt trong code, ảnh chụp, tài liệu hoặc chat.

Repository public. https://hub.docker.com/r/qminh19/voicescope. GitHub chứa source/workflow/docs; Docker Hub chứa image đã build. Docker Hub không tự chạy website hoặc huấn luyện model.

Push main/release/** tạo tag sha-<commit>. Version tag ổn định vMAJOR.MINOR.PATCH tạo thêm tag phiên bản và latest; prerelease không cập nhật latest. v0.2.0 đã tồn tại, không ghi đè. Ví dụ dưới chỉ dùng khi có bản mới đã sẵn sàng:

```
git tag v0.2.1
git push origin v0.2.1
```

Chạy tại checkout đúng commit bạn muốn phát hành. Sau khi CI thành công, xem publish.json trong artifact để lấy tags/digest. Không coi build thành công là push thành công.

Bản public đã xác minh. v0.2.0 và latest cùng trỏ digest bên dưới; tag SHA cũng tồn tại. Đã pull bằng Docker config trống, không đăng nhập, rồi chạy inference và audit chính image tải về.

```
docker pull qminh19/voicescope:v0.2.0
$digest = 'sha256:29fe6d6ec3aea079af51f52fdf7cd10acad9bc12d006d97cabc7d540cdac4d95'
$image = "qminh19/voicescope@$digest"
docker pull $image
```

Digest chọn chính xác artifact; tag là tên tham chiếu và có thể di chuyển. Dùng digest cho triển khai/rollback cần tính xác định. Image public cho phép tải không đăng nhập nhưng vẫn phải tuân thủ giấy phép model CC BY-NC-SA 4.0. Không có dataset hay audio riêng của bạn trong image đã public.

8. Thiết lập CD

Mục đích. Tự triển khai code trên main sau khi CI đạt, rồi xác minh ứng dụng và model đã sẵn sàng. Render build Docker từ source Git; Docker Hub tiếp tục là kênh phát hành image riêng.

Cấu hình đã bật. Dịch vụ voicescope trên Render, gói Free tại Singapore, URL https://voicescope-581u.onrender.com. Branch = main, Auto-Deploy = After CI Checks Pass. Đã đọc lại qua kết nối Render: autoDeploy=yes, autoDeployTrigger=checksPass.

File/cấu hình liên quan. .github/workflows/ci-docker.yml, Dockerfile và voicescope/deployment.py. Workflow có job tests rồi job image kiểm build/inference/audit/publish. /health ưu tiên RENDER_GIT_COMMIT trên Render để không giữ nhầm phiên bản của lần phát hành trước.

Thiết lập trên Dashboard. Trong Settings của dịch vụ, lưu Branch=main và Auto-Deploy=After CI Checks Pass. Chỉ chọn chế độ này khi repo GitHub đã kết nối và main có CI. Nếu dùng Blueprint về sau, đối chiếu template với cấu hình hiện tại trước khi sync để tránh đưa dịch vụ về nhánh hoặc chế độ cũ.

Luồng tự động. Sửa code trong nhánh tính năng, mở PR và xử lý lỗi kiểm thử. Sau khi merge vào main, GitHub Actions chạy test/build/container inference/audit rồi publish image. Render chờ các checks đạt mới build và triển khai cùng commit. Không cần promote feature/deploy-voicescope hay sửa VOICESCOPE_VERSION cho mỗi lần deploy nữa.

```
git switch main
git pull --ff-only origin main
# Tao nhanh tinh nang, sua code va mo PR vao main
```

Chạy Git ở checkout sạch; không đổi nhánh hoặc pull khi còn thay đổi cá nhân chưa xử lý. Đoạn trên chỉ minh họa cách lấy main, không phải lệnh để bỏ qua review hoặc ghi đè code.

Kiểm chứng đầu cuối đã đạt. Commit f50e32975751b167014d94c891d4b665f29939b4 vào main; CI run 41189828565 hoàn tất job image lúc 16:45:13 ngày 31/10/2026 (giờ Việt Nam). Render tự tạo deploy dep-dbirhb7avr4c73bcrnv0 lúc 16:45:16, trigger=new_commit; Live lúc 16:46:13. Không gửi lệnh deploy thủ công. /health.version khớp commit, model_ready=true, HTTP/readiness pass và TTS fake_score=0,99997365 khớp bản kiểm thử. Tag SHA của commit cũng được publish lên Docker Hub.

Kiểm tra sau khi Render báo Live:

```
python -m voicescope.smoke `
  --url https://voicescope-581u.onrender.com `
  --expect ready --require-https
```

Đối chiếu /health.version với commit mà Render đã deploy, /ready 200 và một lượt /inference hợp lệ. Không xem trigger thành công hoặc /health 200 đơn lẻ là deploy thành công. Test TTS chỉ kiểm thực thi; không upload bản ghi riêng của người dùng và không dùng để tuyên bố accuracy.

Rollback và giới hạn. Dashboard > Deploys > chọn bản thành công trước > Rollback nếu artifact còn được giữ. Kiểm lại readiness, version, inference và auto-deploy. Chưa thực hiện rollback production. Docs-only commit có thể dùng [skip render] khi không muốn phát sinh một deploy mới; thay đổi ứng dụng cần CI và deploy bình thường.